

Cash Application Automation

Architecture, AI Design & Implementation Guide

Prepared for Internal Engineering & Treasury Teams
April 2026

1. Problem Overview

Cash Application is the process of matching incoming bank payments to open customer invoices in your ERP system. In B2B environments this is one of the most labour-intensive treasury tasks because:

- Customers send wire transfers or ACH payments with partial, missing, or cryptic remittance information
- Remittance advice emails arrive in dozens of different formats from different customers
- A single payment may cover multiple invoices, include early-pay discounts, or contain short-payments with deductions
- The treasury team must cross-reference bank statements, ERP open invoices, and email inboxes simultaneously

The core question this system answers for every bank credit entry: "Who sent this payment and which invoices does it pay off?"

1.1 What We Are Building

An automated pipeline that ingests bank statement transactions, fetches relevant ERP context, searches for remittance emails, calls an AI model to reason over all evidence, and either auto-posts the match to the ERP or routes it to a human reviewer with the AI suggestion pre-filled.

1.2 Scope

Category	In Scope	Out of Scope (Phase 1)
Inbound payments	Customer wire, ACH, SEPA credits	Outbound vendor payments
Matching logic	Invoice-level matching, deductions, discounts	Disputes, write-offs
Remittance capture	Email body, PDF attachments	AP portal scraping (Phase 2)
ERP posting	Auto-post high-confidence matches	General ledger coding

2. High-Level Architecture

The system has three distinct layers. All sensitive data stays inside your private network. The AI model is called over HTTPS with the minimum context needed — it never receives raw ERP dumps.

LAYER 1 — External Systems (not yours)

Bank SFTP / API | ERP REST API (SAP / NetSuite) | Email Server (Outlook / Gmail)

▼ data pulled by your code

LAYER 2 — Your Private Infrastructure (VPC)

Ingestion Workers (Go/Java) → Message Queue → Local Postgres Cache

Matching Orchestrator (Go/Java) → Confidence Router

Human Review UI (React) + ERP Writer Service

▼ HTTPS API call (prompt + response only)

LAYER 3 — AI Model (external call)

Claude API (Anthropic) | OpenAI API | OR self-hosted open-source model on AWS GPU

Key principle: Your Go/Java services handle ALL data movement. The AI model only sees a small, pre-filtered prompt. It never receives your full ERP database.

3. Where We Use Go/Java vs AI

This is the most important architectural decision. Using AI for everything is expensive and slow. Using only code for everything fails on unstructured data. The split is clear:

Task	Use	Why
Poll bank SFTP / API for new files	Go/Java	Simple file I/O
Call ERP REST API — fetch open invoices	Go/Java	Standard HTTP GET request
Call email API to fetch remittance emails	Go/Java	Microsoft Graph / Gmail API
Fuzzy name matching (bank entry vs customers)	Go/Java	Levenshtein / trigram in code — free, fast
Amount combination search (which invoices sum to payment)	Go/Java	Mathematical search — no AI needed
Exact account number / reference matching	Go/Java	Pure database lookup
Parse structured remittance PDFs (fixed format)	Go/Java	Regex / pdfbox if format is consistent
Extract invoice data from unstructured emails / PDFs	AI	Format varies per customer — AI handles variety
Match payment to invoices (ambiguous cases)	AI	Requires reasoning over evidence
Post approved match back to ERP	Go/Java	Standard ERP REST POST
Store results, write audit logs	Go/Java	Pure database work

The AI only touches two steps: unstructured remittance parsing and ambiguous invoice matching. Everything else is regular code — cheaper, faster, and fully private.

4. The Two-Stage AI Call Chain

Rather than sending all customer data and all invoices in one large expensive prompt, we split the AI work into two small focused calls. Your Go/Java code controls the flow between them.

4.1 Stage 0 — Go/Java Pre-filtering (Free)

Before any AI call, your code narrows 500+ customers down to 3–8 candidates using three cheap filters running against your local database:

1. Fuzzy name match: compare bank entry text (e.g. "ACME CORP INT WIRE") against all customer names using trigram or Levenshtein distance. Returns top matches with a similarity score.
2. Amount combination search: find all customers whose open invoices — individually or in combination — sum to the payment amount, within a 2% tolerance for discounts.
3. Account number / reference lookup: if the bank entry contains an account number or invoice reference, look it up directly in the ERP cache.

4.2 AI Call 1 — Customer Identification (~300 tokens, ~\$0.001)

Input: bank transaction details + the 3–8 pre-filtered candidate customer names and their basic payment patterns. No invoice detail yet.

Output: one customer with a confidence score (0.0–1.0) and a short reasoning note.

Example prompt sent to AI: Bank transaction: \$47,230.00 | Date: 2024-04-10 | Counterparty: "ACME CORP INT WIRE" Candidate customers (pre-filtered by amount and name similarity): 1. Acme Corp Ltd (CUST-0091) — typical wire payer, pays 2-3 invoices together 2. Acme Corporation (CUST-0445) — typical ACH payer 3. Acme Corp International (CUST-0201) — no recent activity Which customer most likely sent this payment? Return JSON: {customer_id, confidence, reasoning}

4.3 Confidence Gate (Go/Java Code)

After Call 1, your code routes the transaction based on the confidence score:

Confidence	Action	Notes
≥ 0.90	Proceed to Call 2 with this customer	Single candidate confirmed
0.70–0.89	Send top 2 candidates to Call 2	Invoice evidence often resolves tie
< 0.70	Skip Call 2 — route to human review	AI not confident enough

4.4 AI Call 2 — Invoice Matching (~600 tokens, ~\$0.002)

Input: confirmed customer + their open invoices + parsed remittance email (if found).

Output: invoice-level match proposal with applied amounts, deductions, confidence score, and reasoning.

```
Example output from AI:{"customer": "Acme Corp Ltd", "confidence": 0.97, "invoices": [ {"id": "INV-312", "applied": 25000, "remaining": 0}, {"id": "INV-287", "applied": 22230, "remaining": 0, "deduction": 500, "reason": "early_pay_discount"}], "unmatched": 0, "reasoning": "Amount matches remittance exactly. $500 deduction within approved policy."}
```

4.5 Final Confidence Router

Confidence	Action	Notes
≥ 0.95 + zero unmatched	Auto-post to ERP	No human needed
0.70–0.94	Human review — AI suggestion pre-filled	One-click approve or adjust
< 0.70	Manual investigation queue	Flag for treasury team

4.6 Short-Circuit Rules (No AI Needed)

A significant portion of transactions — often 30–40% — can be resolved with zero AI calls:

- Exact account number match AND exact amount match → auto-post directly
- Invoice reference number present in bank memo AND matches one open invoice → auto-post directly
- Single customer with single invoice exact amount match → skip Call 1, go to Call 2 only

5. Go/Java Code Structure

5.1 Ingestion Service

Runs on a schedule (e.g. every 4 hours or triggered by bank SFTP arrival). Pulls data from all external sources and normalises it into internal JSON structures stored in Postgres.

```
// BankTransaction — normalised from MT940 / BAI2 / CSV
type BankTransaction struct {
    TxnID          string
    Date           time.Time
    Amount         decimal.Decimal
    Direction      string // 'credit' or 'debit'
    CounterpartyName string
    CounterpartyAcct string
    BankReference  string
    RemittanceInfo string // often blank or cryptic
}

// Fetches open invoices from ERP and caches locally
func SyncERPCache(erpClient *ERPClient, db *sql.DB) error {
    invoices, _ := erpClient.GetOpenInvoices()
    customers, _ := erpClient.GetCustomers()
    // upsert into local postgres
    return upsertAll(db, invoices, customers)
}
```

5.2 Fuzzy Matching in Go

This runs before any AI call. Uses trigram similarity to find customer name candidates from a bank entry description. Zero cost, runs in milliseconds against your local cache.

```
// FuzzyFindCustomers returns top N customers by name similarity
func FuzzyFindCustomers(bankText string, customers []Customer, threshold float64)
[]Candidate {
    var results []Candidate
    normalized := normalise(bankText) // lowercase, strip punctuation
    for _, c := range customers {
        score := trigramSimilarity(normalized, normalise(c.Name))
        if score >= threshold {
            results = append(results, Candidate{Customer: c, Score: score})
        }
    }
    sort.Slice(results, func(i, j int) bool {
        return results[i].Score > results[j].Score
    })
    if len(results) > 8 { results = results[:8] } // cap at 8 candidates
    return results
}
```

5.3 Amount Combination Search

Finds which invoices sum to the payment amount. Uses a recursive subset-sum search with a 2% tolerance for discounts. Pure Go — no AI.

```
// FindInvoiceCombinations returns invoice subsets that sum to target amount
func FindInvoiceCombinations(invoices []Invoice, target decimal.Decimal)
[][]Invoice {
    tolerance := target.Mul(decimal.NewFromFloat(0.02)) // 2% tolerance
    var results [][]Invoice
    var search func(idx int, current []Invoice, sum decimal.Decimal)
    search = func(idx int, current []Invoice, sum decimal.Decimal) {
        diff := target.Sub(sum).Abs()
        if diff.LessThanOrEqual(tolerance) && len(current) > 0 {
            results = append(results, append([]Invoice{}, current...))
        }
        if idx >= len(invoices) { return }
        for i := idx; i < len(invoices); i++ {
            search(i+1, append(current, invoices[i]), sum.Add(invoices[i].Amount))
        }
    }
    search(0, nil, decimal.Zero)
    return results
}
```

5.4 AI API Call (Go)

The matching orchestrator builds the prompt from pre-filtered context and calls the AI API. The base URL is configurable — same code works for Claude, OpenAI, or a self-hosted model on AWS.

```
type AIRequest struct {
    Model    string    `json:"model"`
    Messages []Message `json:"messages"`
}

func CallAI(prompt string) (string, error) {
    body, _ := json.Marshal(AIRequest{
        Model: "claude-sonnet-4-20250514",
        Messages: []Message{{Role: "user", Content: prompt}},
    })
    req, _ := http.NewRequest("POST", os.Getenv("AI_BASE_URL")+"/v1/messages",
bytes.NewBuffer(body))
    req.Header.Set("Content-Type", "application/json")
    req.Header.Set("x-api-key", os.Getenv("AI_API_KEY"))
    resp, err := http.DefaultClient.Do(req)
    // parse response and return text content
    ...
}

// To switch to self-hosted model: set AI_BASE_URL=http://your-gpu-server:8000
```



```
// No other code changes needed - OpenAI-compatible endpoint format
```

6. Infrastructure Options

There are three infrastructure models. The right choice depends on your data privacy requirements and transaction volume.

Factor	Claude API	AWS GPU (self-hosted)	On-Premise
Upfront cost	\$0	\$0	\$5k – \$25k hardware
Monthly cost (500 txns/day)	~\$15	\$60 – \$800	~\$50 – \$100 electricity
Data leaves your network?	Yes (HTTPS to Anthropic)	No — stays in your AWS VPC	No — stays on your servers
Data on AWS servers?	No	Yes (you rent their hardware)	No
Match quality	Best (Claude/GPT-4)	Good (~88% of Claude)	Good (~88% of Claude)
Setup effort	Near zero	1–2 weeks	High
Maintenance	Zero	Low (AWS manages HW)	High (you own everything)
3-year total cost	~\$540	~\$18k – \$30k	~\$10k – \$28k
Best for	Getting started, low volume	Privacy rules, no HW budget	Strict data residency laws

Recommendation: Start with the Claude API at ~\$15/month. Validate the matching logic with real data. Switch to AWS self-hosted only if privacy requirements or volume growth make it necessary.

6.1 AWS Batch Processing Cost Optimisation

Bank statements typically arrive once per day. You do not need your GPU instance running 24/7. Spin it up for the processing window and shut it down afterwards:

- Bank file arrives at 06:00 — AWS instance starts automatically
- 500 transactions processed in approximately 2 hours
- Instance shuts down — you pay for 2 hours only
- Cost: g5.xlarge at \$1.01/hour × 2 hours × 30 days = ~\$60/month instead of ~\$720/month continuous

AWS Spot Instances give a further 70–90% discount for batch workloads that are not time-sensitive.

6.2 Your App Infrastructure (Separate From GPU)

These services run on normal inexpensive servers and are needed regardless of which AI option you choose:

Component	Technology	Monthly cost (AWS)
App servers (Go/Java services)	2–3 × t3.medium EC2	~\$80 – \$120
Database	Postgres RDS db.t3.small	~\$50 – \$80
Message queue	SQS	~\$2 (near free at this volume)
Human review UI	React app on S3 + CloudFront	~\$5
Total app infrastructure		~\$140 – \$210 / month

7. Data Privacy & Security

7.1 What Data Leaves Your Network

When using the Claude or OpenAI API, only the AI prompt travels outside your network over HTTPS. This prompt contains:

- 3–8 customer names and their payment patterns (pre-filtered candidates only — not all 500 customers)
- Open invoice numbers and amounts for the matched customer only
- Parsed remittance data (invoice numbers, amounts, dates)
- Bank transaction amount and counterparty name

The following never leaves your network:

- Full customer database or ERP data
- Bank account numbers or IBAN details
- Employee data, pricing, or contracts
- Historical transaction archives

7.2 Model Training — Your Data Will Not Be Used

This is a common and important concern. The contractual position for both main providers is:

Provider	Default API policy	What to request
Anthropic (Claude)	API data not used for training by default	Sign DPA + enable Zero Data Retention
OpenAI	API data not used for training by default	Sign DPA + enable zero-retention mode
Self-hosted on AWS	Your model, your data — no third party AI provider involved	AWS Data Processing Addendum

Action item: Before going to production, request and sign a Data Processing Agreement (DPA) with your chosen AI provider and enable Zero Data Retention on your API account. Both are standard enterprise options that take less than a day to set up.

7.3 Additional Security Measures

- Deploy all services inside a private VPC with no public internet access except the AI API outbound call
- Encrypt all data at rest in Postgres (AWS RDS encryption is one checkbox)
- Encrypt all data in transit (TLS 1.3 on all internal service calls)
- Store AI API keys in AWS Secrets Manager — never in code or environment files
- Log every AI prompt and response in your audit table for financial compliance

- Role-based access: treasury team can see the review UI; only the ERP writer service can post to ERP

8. Pretrained & Open Source Models

There are two categories of models available: specialized document understanding models (for remittance parsing) and general reasoning LLMs (for matching logic). You do not need to train anything from scratch.

8.1 Document Understanding Models — Remittance Parsing

These small specialized models are pretrained specifically to extract structured fields from financial documents. They run on CPU — no GPU required — and are completely free.

Model	Developer	Hardware	Best for
LayoutLMv3	Microsoft	CPU only	Invoice PDFs — understands layout and text together
Donut	Naver	CPU only	Invoice images — no OCR step needed, outputs JSON directly
LayoutLM-invoices	Impira	CPU only	Already fine-tuned on real invoices — use as-is from HuggingFace

Recommendation: Use LayoutLM-invoices (huggingface.co/impira/layoutlm-invoices) for remittance PDF parsing. It is pretrained on real invoices and may eliminate the need for an LLM on this step entirely, saving significant cost.

8.2 Open Source LLMs for Matching Logic

For the reasoning steps (Call 1 and Call 2), these open source models can be self-hosted on AWS GPU instances. They are free to download and use commercially.

Model	Size	GPU needed	Quality vs Claude	Best for
Phi-4	14B	1× RTX 4090 (24GB)	~85%	Call 1 — customer ID
Qwen2.5	32B	1× A100 or 2× RTX 4090	~88%	Call 2 — invoice match
LLM Pro Finance 70B	70B	2× A100 48GB	~92%	Best quality, finance-tuned
QwQ-32B	32B	1× A100 or 2× RTX 4090	~88%	Strong math and reasoning
Llama 3.1	8B	1× RTX 3090 (24GB)	~75%	Budget option, limited reasoning

8.3 Finance-Specific LLMs

DragonLLM and AGEFI released the LLM Open Finance suite in late 2025 — open source models fine-tuned specifically on financial language tasks. Available at huggingface.co/collections/DragonLLM/llm-open-finance. These models outperform general-purpose LLMs on financial text tasks including RAG, classification, and financial reasoning.

8.4 Running a Model on AWS (Self-Hosted)

Using vLLM as the inference server. Your Go/Java code does not change — vLLM exposes the same REST endpoint format as OpenAI.

```
# On your AWS GPU instance — install vLLM
pip install vllm

# Start the model server (Qwen2.5 32B quantized)
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen2.5-32B-Instruct \
  --quantization awq \
  --max-model-len 8192 \
  --host 0.0.0.0 \
  --port 8000

# Your Go service now points to this instead of Claude API:
# AI_BASE_URL=http://your-gpu-instance:8000
# No API key needed — it is your own server
```

Using Ollama is an even simpler alternative to vLLM for getting started: `curl -fsSL https://ollama.ai/install.sh | sh` `ollama pull llama3.1:8b` # Model now running at `http://localhost:11434` — OpenAI-compatible API

9. Cost Summary

9.1 AI API Costs (Claude — Recommended Starting Point)

Claude Sonnet is priced at approximately \$3 per million input tokens and \$15 per million output tokens. Each matching transaction uses roughly 900 tokens across both calls.

Transaction type	AI calls	Tokens	Cost per txn
Exact match — no AI needed	0	0	Free
Single customer — skip Call 1	1 (Call 2)	~600	~\$0.002
Full two-call chain	2	~900	~\$0.003
Low confidence — Call 1 only	1 (stops)	~300	~\$0.001
Average across 500 txns/day	~0.8	~500	~\$0.0015 avg

Period	Estimated AI cost
Per day (500 transactions)	~\$0.35 – \$0.75
Per month	~\$10 – \$22
Per year	~\$120 – \$265

9.2 Total System Cost — Claude API Option

Component	Monthly cost
App servers (3× t3.medium)	~\$90
Postgres RDS	~\$60
SQS + misc AWS	~\$10
Claude API (AI matching)	~\$15
Human review UI hosting	~\$5
TOTAL	~\$180 / month

A single treasury analyst costs \$4,000–\$8,000/month in salary. This system at \$180/month automates 70–90% of their matching workload, reducing the team to exception handling only.

10. Implementation Roadmap

Phase	Timeline	Deliverables
1 — Proof of Concept	Weeks 1–2	Take 20 historical matched transactions. Build context builder and prompt. Measure AI accuracy manually. No infrastructure yet.
2 — Remittance Parser	Weeks 3–4	Connect to email API. Deploy LayoutLM-invoices for PDF parsing. Test on real customer emails.
3 — Full Pipeline	Weeks 5–6	Wire all stages together. Build confidence router. Deploy with 100% human review but AI suggestions pre-filled.
4 — Auto-post Ramp	Month 2	Enable auto-post for ≥ 0.95 confidence. Track false positive rate. Tune confidence thresholds.
5 — Fine-Tuning	Month 6+	Use 6 months of labeled match history to fine-tune a smaller open source model on your specific customer patterns.

Most important advice: log every match decision from day one — the AI's reasoning, the confidence score, whether it was correct or not. This data is what you will use to fine-tune a model later and it becomes more valuable over time.

10.1 Technology Stack Summary

Layer	Technology	Notes
Primary language	Go or Java	Your team's preference — both work equally well
AI model — Phase 1	Claude API (claude-sonnet-4)	Best quality, minimal setup, ~\$15/month
AI model — Phase 2	Qwen2.5 32B on AWS g5 GPU	If data privacy requires fully self-hosted
Remittance parsing	LayoutLM-invoices (HuggingFace)	Free, CPU-only, pretrained on invoices
Database	Postgres (AWS RDS)	Local ERP cache + match audit log
Queue	AWS SQS	Decouples ingestion from matching
Model serving (self-hosted)	vLLM or Ollama	OpenAI-compatible API — one config change
Human review UI	React	Shows AI suggestion, approve / adjust / reject